**Caso de estudio: NeoMarket — Marketplace omnicanal API-driven**

**1. Contexto y Desafíos de Ingeniería**

NeoMarket enfrenta una transición crítica hacia una arquitectura API-First debido a tres frentes operativos de alta exigencia:

- Omnicanalidad Extrema: Clientes concurrentes desde la Web (SSR), App Móvil (iOS/Android) y sistemas POS de Tiendas Partner físicas.
- Ofertas Flash (Picos de Tráfico): Campañas de descuento agresivas que generan incrementos repentinos de tráfico de hasta un 1200% en consultas y transacciones de inventario.
- Programa de Afiliados (API Pública): Apertura de endpoints para que terceros listen productos y redirijan tráfico, requiriendo un control estricto de cuotas (*Rate Limiting*) y seguridad.

Deben elegir la(s) arquitectura(s) orientadas a APIs adecuadas**.**

**2) Requerimientos (funcionales y no funcionales)**

**Funcionales**

- Catálogo: listar/buscar productos, precios dinámicos, stock por tienda.
- Carrito y checkout: cupones, impuestos, múltiples métodos de pago.
- Pedidos: creación, actualización de estado, devoluciones.
- Partners: alta/baja de tiendas, webhooks para cambios de pedidos.
- Afiliados: tracking de referidos, reportes
- Notificaciones: email, push, SMS.

**No funcionales**

- **Escalabilidad**: picos de 1k→25k RPS(solicitudes por segundo) en ofertas flash.
- **Latencia**: P95 < 250 ms lectura catálogo; P95 < 400 ms checkout.
- **Disponibilidad**: 99.95% core (checkout/pedidos).
- **Seguridad**: OAuth2/OIDC, mTLS interno, rate limiting.
- **Observabilidad**: logs estructurados, traces (OpenTelemetry), métricas (RPS, latencia, error rate).

**3) Stakeholders**

- Producto, Ingeniería (plataforma y feature teams), Partners externos, Compliance, Soporte/SRE, Marketing (campañas).

**4) Datos y contratos (extracto)**

**Producto (JSON)**

{ "id":"SKU-123", "nombre":"Zapatillas", "precio":59.90, "stock":12, "tienda_id":"T-001", "tags":["running","sale"] }

**Pedido (JSON)**

{ "id":"ORD-9001", "items":[{"sku":"SKU-123","qty":1}], "estado":"CREATED", "total":59.90 }

**Evento (Avro) order.status.changed**

{ orderId: string, oldStatus: string, newStatus: string, at: long }

**5) Escenarios clave a resolver**

1. **Lectura intensiva** de catálogo (alto caché/CDN).
2. **Checkout** sensible a latencia, con reglas de negocio.
3. **Actualizaciones de estado** de pedido hacia apps y partners en **tiempo casi real**.
4. **Integración de partners** heterogéneos (algunos solo soportan webhooks).
5. **Reportería/analytics** sin impactar al OLTP.